



POLITECNICO
MILANO 1863

Driver for Adafruit Neo Trinkey - SAMD21 USB Key

Samuele Lo Piccolo



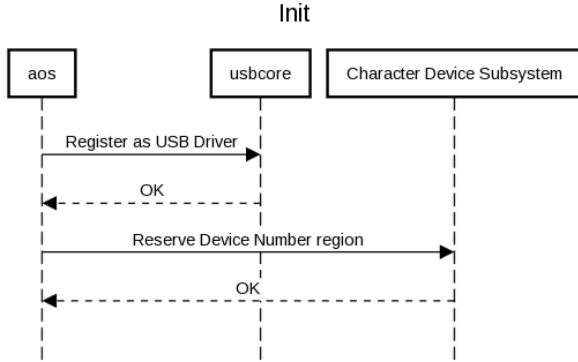
POLITECNICO
MILANO 1863

Main Purpose of this Project



The objective was communication with a USB-CDC serial interface provided by CircuitPython, the REPL, to be able to control the LEDs on the device from a Linux host instead of making a static Circuit Python program.

Implementation of the Driver – Init and Probe

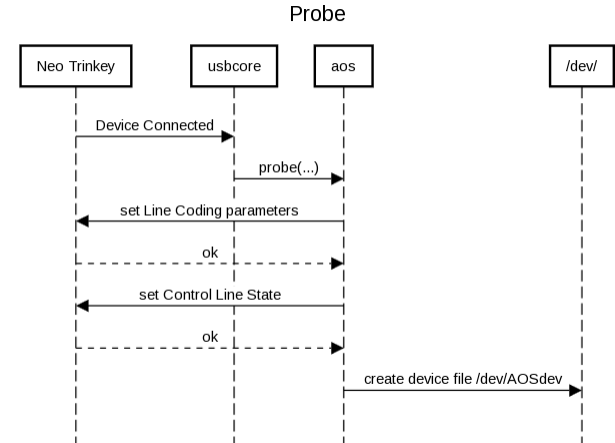


The driver registers itself with USBcore, exporting the device table and exposing the Probe function.

Then it reserves the character device region, needed to create a device file when the device connects.

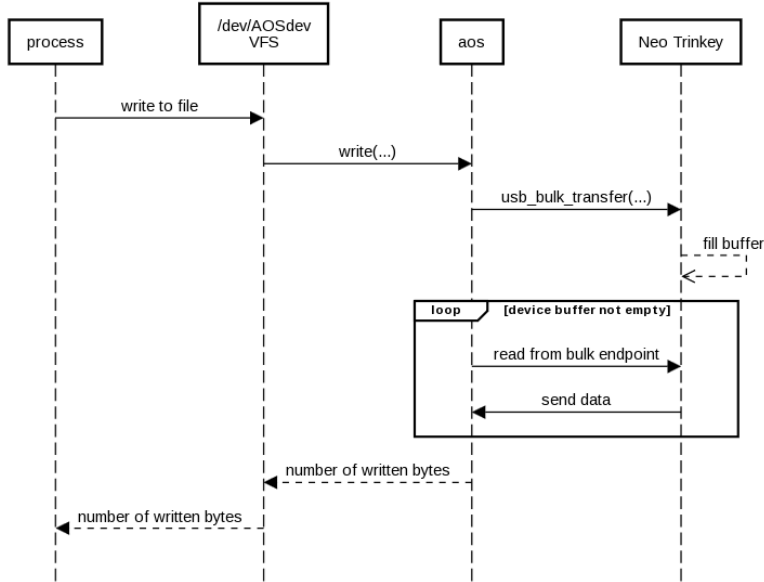
The **Probe** function sets up the CDC interface of the Neo Trinkey for bulk transfers by setting the Line Coding parameters and the Control Line State

It also creates the device file in `/dev/`



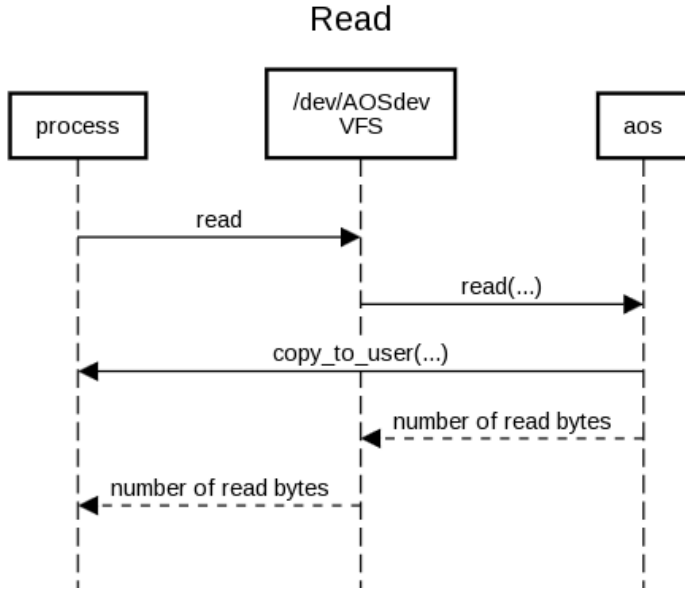
Implementation of the Driver - Write

Write



The **Write** function transfers up to 64 bytes at a time to the device, then it empties a buffer present on the device to make sure no write gets ignored when the buffer is full.

Implementation of the Driver - Read



The **Read** function does not interact with the Neo Trinkey at all since it's the write function that has the duty of keeping the device buffer empty to avoid problems.

It just accesses a local buffer that contains the data produced by the device and returns it to userspace; this local buffer is flushed if it ever overflows or if every character is read.

Validation

My main approach to test the correctness of the project was to use Linux command line tools to communicate through `/dev/A0Sdev` and verifying that the LEDs behaved as intended.

```
> printf "\rimport board\rimport neopixel\rpixels = neopixel.NeoPixel(board.NEOPIXEL, 4)\rpixels.fill((0x220000))\r">/dev/A0Sdev
```

```
> cat /dev/A0Sdev
Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.

Press any key to enter the REPL. Use CTRL-D to reload.

Adafruit CircuitPython 10.1.3 on 2026-02-20; Adafruit NeoPixel Trinkey M0 with samd21e18
>>> import board
>>> import neopixel
>>> pixels = neopixel.NeoPixel(board.NEOPIXEL, 4)
>>> pixels.fill((0x220000))
>>>
```

In the pictures I show the commands needed to turn the LEDs red and the REPL output.

Problems Encountered

The main problem was figuring out some device quirks without documentation.

```
> printf "\rimport board\rimport neopixel\rpixels = neopixel.NeoPixel(board.NEOPIXEL, 4)\r\n\r\npixels.fill((0x220000))\r\n">/dev/A0Sdev
```

For instance here I use `\r` because that's the expected newline instead of `\n`

The biggest problem was figuring out why sometimes the device stopped accepting writes after a while.

I had to go by trial and error, but I managed to understand that the device was waiting for me to read from it before it could accept any new writes.

Learning Outcomes

I learned how USB descriptors work and how to identify the ones I needed to use for my project.

I have also gained a better understanding of how to work in a Linux environment and of the Linux kernel module development workflow.

```
Interface Descriptor:
  bLength: 9
  bDescriptorType: 4
  bInterfaceNumber: 1
  bAlternateSetting: 0
  bNumEndpoints: 2
  bInterfaceClass: 10 CDC Data
  bInterfaceSubClass: 0 [unknown]
  bInterfaceProtocol: 0
  iInterface: 5
Endpoint Descriptor:
  bLength: 7
  bDescriptorType: 5
  bEndpointAddress: 0x02 EP 2 OUT
  bmAttributes: 2
    Transfer Type: Bulk
    Synch Type: None
    Usage Type: Data
  wMaxPacketSize: 0x0040 1x 64 bytes
  bInterval: 0
Endpoint Descriptor:
  bLength: 7
  bDescriptorType: 5
  bEndpointAddress: 0x82 EP 2 IN
  bmAttributes: 2
    Transfer Type: Bulk
    Synch Type: None
    Usage Type: Data
  wMaxPacketSize: 0x0040 1x 64 bytes
  bInterval: 0
```